

06 Feature Engineering and Transformation

Course Name: Data Management for Machine Learning
Assignment Title: End-to-End Data Management Pipeline for a Recommendation System
Assignment: Group 51 - Data management for Machine Learning Group 51

Team Members

Team Member Name	Team Member ID
BANSHIDHAR RATH	2025AE05346
JITENDRA KUMAR TIWARI	2025AE05518
KATBA ANKIT CHIMANBHAI	2025AE05229
NAVEEN SURATHU	2025AE05492

1. Objective

This report documents the feature engineering and transformation stage for the recommendation pipeline. It captures feature creation logic, transformation assets, SQL warehouse schema, and transformed outputs stored in a structured database layer.

2. Objective Coverage

- Create recommendation-ready features such as user activity frequency.
- Compute average rating style features per user and item.
- Build co-occurrence or similarity-style item pair features.
- Store transformed outputs in a structured warehouse database.
- Provide SQL schema, transformation script evidence, and summary of feature logic.

3. Supporting Project and Log Artifacts

The following discovered files support this section, including source datasets, validation outputs, and generated warehouse assets. Long paths are wrapped inside table cells using real line breaks at safe separators only.

Artifact	Relative Path	Last Modified	Size
events.csv	data\raw\retailrocket\load_date=2026-04-29\load_hour=11\events.csv	2026-04-20 17:28:00	92029.21 KB
prepared_interactions_20260429_215532.parquet	data\prepared\prepared_interactions_20260429_215532.parquet	2026-04-29 21:55:34	65323.8 KB
products.parquet	data\bronze\dummyjson\products.parquet	2026-04-29 16:42:35	73.89 KB
run_validation.txt	src\02-validation\run_validation.txt	2026-04-28 20:14:12	46.94 KB
data_quality_report_20260429T111342Z.json	reports\validation\data_quality_report_20260429T111342Z.json	2026-04-29 16:47:43	6.66 KB

Artifact	Relative Path	Last Modified	Size
data_quality_report_20260429T111342Z.pdf	reports\validation\data_quality_report_20260429T111342Z.pdf	2026-04-29 16:47:44	8.49 KB
validation_log_20260429T111342Z.jsonl	logs\validation_log_20260429T111342Z.jsonl	2026-04-29 16:47:44	0.66 KB
recommendation_features.db	warehouse\recommendation_features.db	2026-04-29 22:06:45	1553084.0 KB
recommendation_features_schema.sql	warehouse\recommendation_features_schema.sql	2026-04-29 22:07:21	1.09 KB

4. Transformation Scripts and SQL Assets

Transformation Asset	Relative Path	Last Modified	Size
materialize_features_executed.ipynb	runs\prefect\20260429_160933\executed_notebooks\materialize_features\materialize_features_executed.ipynb	2026-04-29 16:25:43	17.27 KB
materialize_features_executed.ipynb	runs\prefect\20260429_164131\executed_notebooks\materialize_features\materialize_features_executed.ipynb	2026-04-29 17:05:20	17.27 KB
materialize_features.ipynb	src\04-features\materialize_features.ipynb	2026-04-29 15:22:54	15.82 KB
06 Feature Engineering and Transformation.ipynb	src\07-submission\06 Feature Engineering and Transformation.ipynb	2026-04-29 22:02:03	55.34 KB

5. Feature Logic Summary

- Standardized common interaction columns where present.
- Dropped rows missing required interaction keys: 0 removed.
- Removed duplicate interactions using user_id, item_id, event_type, event_ts: 540219 removed.
- Standardized common product columns where present.
- Prepared product price fields for downstream feature joins.
- Removed duplicate product_id rows: 0 removed.
- User activity frequency = count of interactions per user.
- No explicit rating column was available, so weighted interaction score was used as a proxy average rating.
- Item popularity = count of interactions and distinct users per item.
- User-item features = interaction count, total interaction weight, latest event timestamp, and event-type counts.
- Co-occurrence features = pair counts for items appearing under the same user history.
- Transformed tables are persisted to a structured SQLite warehouse for reporting and downstream use.

6. Structured Warehouse Outputs

Transformed Output	Relative Path	Type	Last Modified	Size
user_features.csv	data\transformed\user_features.csv	.csv	2026-04-29 22:06:51	28118.81 KB

Transformed Output	Relative Path	Type	Last Modified	Size
item_features.csv	data\transformed\item_features.csv	.csv	2026-04-29 22:06:51	6078.2 KB
user_item_features.csv	data\transformed\user_item_features.csv	.csv	2026-04-29 22:06:56	58601.84 KB
item_cooccurrence.csv	data\transformed\item_cooccurrence.csv	.csv	2026-04-29 22:07:21	523140.67 KB
recommendation_features.db	warehouse\recommendation_features.db	.db	2026-04-29 22:06:45	1553084.0 KB
recommendation_features_schema.sql	warehouse\recommendation_features_schema.sql	.sql	2026-04-29 22:07:21	1.09 KB

7. Warehouse and Transformed Folder Structure

warehouse/

```
warehouse/
  recommendation_features.db
  recommendation_features_schema.sql
```

data/transformed/

```
transformed/
  item_cooccurrence.csv
  item_features.csv
  user_features.csv
  user_item_features.csv
```

8. Feature Table Summary

table_name	rows	columns	duplicate_rows
user_features	1407580	5	0
item_features	235061	9	0
user_item_features	2145179	8	0
item_cooccurrence	32411667	3	0

9. User Features Preview

user_id	activity_frequency	unique_items_interacted	avg_interaction_weight	avg_rating_per_user
0	3	3	1.0	1.0
1	1	1	1.0	1.0
2	4	4	1.0	1.0
3	1	1	1.0	1.0
4	1	1	1.0	1.0
5	1	1	1.0	1.0
6	3	3	1.6666666666666667	1.6666666666666667
7	3	3	1.0	1.0
8	1	1	1.0	1.0
9	1	1	1.0	1.0
10	1	1	1.0	1.0
11	1	1	1.0	1.0
12	1	1	1.0	1.0
13	2	2	1.0	1.0
14	1	1	1.0	1.0

10. Item Features Preview

item_id	item_interaction_count	unique_users	avg_interaction_weight	avg_rating_per_item	title	category	price	price_norm
3	2	2	1.0	1.0	Powder Canister	beauty	14.99	0.00038379208199096197
4	3	3	1.0	1.0	Red Lipstick	beauty	12.99	0.00032973685917533354
6	26	26	1.0	1.0	Calvin Klein CK One	fragrances	49.99	0.00132975848126446
9	2	2	1.0	1.0	Dolce Shine Eau de	fragrances	69.99	0.001870310709420744

item_id	item_interaction_count	unique_users	avg_interaction_weight	avg_rating_per_item	title	category	price	price_norm
15	16	13	1.5	1.5	Wooden Bathroom Sink With Mirror	furniture	799.99	0.02160046703712513
16	14	14	1.0	1.0	Apple	groceries	1.99	3.2433133689377066e-05
17	2	2	1.0	1.0	Beef Steak	groceries	12.99	0.00032973685917533354
19	16	15	1.375	1.375	Chicken Meat	groceries	9.99	0.0002486540249518909
22	2	2	1.0	1.0	Dog Food	groceries	10.99	0.00027568163635970506
24	1	1	1.0	1.0	Fish Steak	groceries	14.99	0.00038379208199096197
25	54	49	1.2962962962962963	1.2962962962962963	Green Bell Pepper	groceries	1.29	1.3513805703907113e-05
26	18	18	1.0	1.0	Green Chili Pepper	groceries	0.99	5.405522281562844e-06
29	9	9	1.0	1.0	Juice	groceries	3.99	8.648835650500552e-05
32	55	55	1.0	1.0	Milk	groceries	3.49	7.297455080109841e-05
33	11	11	1.3636363636363635	1.3636363636363635	Mulberry	groceries	4.99	0.00011351596791281974

11. User-Item Features Preview

user_id	item_id	interaction_count	total_interaction_weight	last_event_ts	addtocart	transaction	view
0	67045	1	1.0		0	0	1
0	285930	1	1.0		0	0	1
0	357564	1	1.0		0	0	1
1	72028	1	1.0		0	0	1
2	216305	1	1.0		0	0	1
2	259884	1	1.0		0	0	1
2	325215	1	1.0		0	0	1
2	342816	1	1.0		0	0	1
3	385090	1	1.0		0	0	1
4	177677	1	1.0		0	0	1
5	61396	1	1.0		0	0	1
6	65273	1	3.0		1	0	0
6	253615	1	1.0		0	0	1
6	344723	1	1.0		0	0	1
7	139394	1	1.0		0	0	1

12. Item Co-occurrence Preview

item_id_a	item_id_b	cooccurrence_count
218794	461686	259
219512	384302	226
257040	309778	197
171878	461686	194
180751	298009	184
241555	9877	162
340093	96924	146
109618	243355	139
370653	84160	139
171878	218794	130
241555	325310	129
325310	9877	129
335975	370653	129
354759	452896	128
38965	9877	122

13. SQL Schema

The SQL schema below was exported from the structured SQLite warehouse after the transformed tables were created.

```
CREATE INDEX idx_item_cooccurrence_pair ON item_cooccurrence(item_id_a, item_id_b);

CREATE INDEX idx_item_features_item_id ON item_features(item_id);

CREATE INDEX idx_user_features_user_id ON user_features(user_id);

CREATE INDEX idx_user_item_user_item ON user_item_features(user_id, item_id);

CREATE TABLE "item_cooccurrence" (
  "item_id_a" TEXT,
  "item_id_b" TEXT,
  "cooccurrence_count" INTEGER
);

CREATE TABLE "item_features" (
  "item_id" TEXT,
  "item_interaction_count" INTEGER,
  "unique_users" INTEGER,
  "avg_interaction_weight" REAL,
  "avg_rating_per_item" REAL,
  "title" TEXT,
  "category" TEXT,
  "price" REAL,
  "price_norm" REAL
);

CREATE TABLE "user_features" (
  "user_id" INTEGER,
  "activity_frequency" INTEGER,
  "unique_items_interacted" INTEGER,
  "avg_interaction_weight" REAL,
  "avg_rating_per_user" REAL
);

CREATE TABLE "user_item_features" (
  "user_id" INTEGER,
  "item_id" INTEGER,
  "interaction_count" INTEGER,
  "total_interaction_weight" REAL,
  "last_event_ts" TIMESTAMP,
  "addtocart" INTEGER,
  "transaction" INTEGER,
  "view" INTEGER
);
```

14. Transformation Code / Asset Preview

Asset: *materialize_features_executed.ipynb*

Path:

runs\prefect\20260429_160933\executed_notebooks\materialize_features\materialize_features_executed.ipynb

```
# Code cell 1
from __future__ import annotations

from pathlib import Path
import numpy as np
import pandas as pd

# -----
# Project paths and setup
# -----
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent
#PROJECT_ROOT = Path(__file__).resolve().parents[1]
PREP_ROOT = PROJECT_ROOT / "data" / "prepared"
FEATURE_ROOT = PROJECT_ROOT / "data" / "featured" / "v1"

INTERACTIONS_PATH = PREP_ROOT / "interactions_prepared.csv"
PRODUCTS_PATH = PREP_ROOT / "products_prepared.csv"

USER_FEATURES_PATH = FEATURE_ROOT / "user_features.parquet"
ITEM_FEATURES_PATH = FEATURE_ROOT / "item_features.parquet"
INTERACTION_FEATURES_PATH = FEATURE_ROOT / "interaction_features.parquet"

def min_max_scale(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors="coerce")
    if s.notna().sum() == 0:
        return pd.Series(np.nan, index=series.index)

    min_val = s.min()
    max_val = s.max()

    if pd.isna(min_val) or pd.isna(max_val):
        return pd.Series(np.nan, index=series.index)

    if min_val == max_val:
        return pd.Series(0.0, index=series.index)

    return (s - min_val) / (max_val - min_val)

def ensure_required_columns(df: pd.DataFrame, required: list[str], df_name: str) -> None:
    missing = [c for c in required if c not in df.columns]
    if missing:
        raise ValueError(f"{df_name} is missing required columns: {missing}")

def load_prepared_data() -> tuple[pd.DataFrame, pd.DataFrame]:
    if not INTERACTIONS_PATH.exists():
        raise FileNotFoundError(f"Missing file: {INTERACTIONS_PATH}")

    if not PRODUCTS_PATH.exists():
        raise FileNotFoundError(f"Missing file: {PRODUCTS_PATH}")

    interactions = pd.read_csv(INTERACTIONS_PATH)
    products = pd.read_csv(PRODUCTS_PATH)

    ensure_required_columns(
        interactions,
        ["user_id", "item_id", "event", "event_weight", "event_ts", "event_date"],
        "interactions_prepared.csv",
    )
    ensure_required_columns(
        products,
        ["category"],
        "products_prepared.csv",
    )

    interactions["user_id"] = pd.to_numeric(interactions["user_id"],
errors="coerce").astype("Int64")
    interactions["item_id"] = pd.to_numeric(interactions["item_id"],
errors="coerce").astype("Int64")
    interactions["event"] = interactions["event"].astype(str).str.strip().str.lower()
    interactions["event_weight"] = pd.to_numeric(interactions["event_weight"], errors="coerce")
    interactions["event_ts"] = pd.to_datetime(interactions["event_ts"], errors="coerce")
    interactions["event_date"] = pd.to_datetime(interactions["event_date"], errors="coerce")

    interactions = interactions.dropna(subset=["user_id", "item_id", "event", "event_weight",
"event_ts"])
```



```

interactions["user_id"] = interactions["user_id"].astype("int64")
interactions["item_id"] = interactions["item_id"].astype("int64")

products.columns = [c.strip() for c in products.columns]

return interactions, products

def normalize_products(products: pd.DataFrame) -> pd.DataFrame:
    products = products.copy()

    if "item_id" not in products.columns:
        if "product_id" in products.columns:
            products["item_id"] = pd.to_numeric(products["product_id"], errors="coerce")
        elif "id" in products.columns:
            products["item_id"] = pd.to_numeric(products["id"], errors="coerce")

    if "item_id" not in products.columns:
        raise ValueError("products_prepared.csv must contain one of: item_id, product_id, or
id")

    products["item_id"] = pd.to_numeric(products["item_id"], errors="coerce")
    products = products.dropna(subset=["item_id"]).copy()
    products["item_id"] = products["item_id"].astype("int64")

    if "category" in products.columns:
        products["category"] = (
            products["category"]
            .astype(str)
            .str.strip()
            .str.lower()
        )
    else:
        products["category"] = "unknown"

    if "price_norm" not in products.columns:
        products["price_norm"] = min_max_scale(products["price"]) if "price" in
products.columns else 0.0

    if "rating_norm" not in products.columns:
        if "rating" in products.columns:
            products["rating_norm"] = min_max_scale(products["rating"])
        else:
            products["rating_norm"] = 0.0

    if "stock_norm" not in products.columns:
        if "stock" in products.columns:
            products["stock_norm"] = min_max_scale(products["stock"])
        else:
            products["stock_norm"] = 0.0

    keep_cols = ["item_id", "price_norm", "rating_norm", "stock_norm", "category"]
    return products[keep_cols].drop_duplicates(subset=["item_id"]).reset_index(drop=True)

def build_user_features(interactions: pd.DataFrame) -> pd.DataFrame:
    max_ts = interactions["event_ts"].max()

    user_features = (
        interactions.groupby("user_id")
        .agg(
            user_total_interactions=("user_id", "size"),
            user_view_count=("event", lambda s: (s == "view").sum()),
            user_addtocart_count=("event", lambda s: (s == "addtocart").sum()),
            user_transaction_count=("event", lambda s: (s == "transaction").sum()),
            user_avg_event_weight=("event_weight", "mean"),
            user_active_days=("event_date", lambda s: s.dt.date.nunique()),
            last_event_ts=("event_ts", "max"),
        )
        .reset_index()
    )

    user_features["user_last_seen_recency_days"] = (
        (max_ts - user_features["last_event_ts"]).dt.total_seconds() / 86400.0
    )

    user_features = user_features.drop(columns=["last_event_ts"])

    return user_features

def build_item_features(interactions: pd.DataFrame, products: pd.DataFrame) -> pd.DataFrame:
    item_features = (
        interactions.groupby("item_id")
        .agg(
            item_total_interactions=("item_id", "size"),
            item_view_count=("event", lambda s: (s == "view").sum()),
            item_addtocart_count=("event", lambda s: (s == "addtocart").sum()),
            item_transaction_count=("event", lambda s: (s == "transaction").sum()),

```

```

        item_avg_event_weight=("event_weight", "mean"),
    )
    .reset_index()
)

item_features["item_popularity_rank"] = (
    item_features["item_total_interactions"]
    .rank(method="dense", ascending=False)
    .astype("int64")
)

products_norm = normalize_products(products)

item_features = item_features.merge(products_norm, on="item_id", how="left")

item_features["category"] = item_features["category"].fillna("unknown")
for col in ["price_norm", "rating_norm", "stock_norm"]:
    item_features[col] = item_features[col].fillna(0.0)

return item_features

def build_interaction_features(interactions: pd.DataFrame) -> pd.DataFrame:
    max_ts = interactions["event_ts"].max()

    interaction_features = (

```

Asset: materialize_features_executed.ipynb

Path:

runs\prefect\20260429_164131\executed_notebooks\materialize_features\materialize_features_executed.ipynb

```

# Code cell 1
from __future__ import annotations

from pathlib import Path
import numpy as np
import pandas as pd

# -----
# Project paths and setup
# -----
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent
#PROJECT_ROOT = Path(__file__).resolve().parents[1]
PREP_ROOT = PROJECT_ROOT / "data" / "prepared"
FEATURE_ROOT = PROJECT_ROOT / "data" / "featured" / "v1"

INTERACTIONS_PATH = PREP_ROOT / "interactions_prepared.csv"
PRODUCTS_PATH = PREP_ROOT / "products_prepared.csv"

USER_FEATURES_PATH = FEATURE_ROOT / "user_features.parquet"
ITEM_FEATURES_PATH = FEATURE_ROOT / "item_features.parquet"
INTERACTION_FEATURES_PATH = FEATURE_ROOT / "interaction_features.parquet"

def min_max_scale(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors="coerce")
    if s.notna().sum() == 0:
        return pd.Series(np.nan, index=series.index)

    min_val = s.min()
    max_val = s.max()

    if pd.isna(min_val) or pd.isna(max_val):
        return pd.Series(np.nan, index=series.index)

    if min_val == max_val:
        return pd.Series(0.0, index=series.index)

    return (s - min_val) / (max_val - min_val)

def ensure_required_columns(df: pd.DataFrame, required: list[str], df_name: str) -> None:
    missing = [c for c in required if c not in df.columns]
    if missing:
        raise ValueError(f"{df_name} is missing required columns: {missing}")

def load_prepared_data() -> tuple[pd.DataFrame, pd.DataFrame]:
    if not INTERACTIONS_PATH.exists():
        raise FileNotFoundError(f"Missing file: {INTERACTIONS_PATH}")

    if not PRODUCTS_PATH.exists():
        raise FileNotFoundError(f"Missing file: {PRODUCTS_PATH}")

```

```

interactions = pd.read_csv(INTERACTIONS_PATH)
products = pd.read_csv(PRODUCTS_PATH)

ensure_required_columns(
    interactions,
    ["user_id", "item_id", "event", "event_weight", "event_ts", "event_date"],
    "interactions_prepared.csv",
)
ensure_required_columns(
    products,
    ["category"],
    "products_prepared.csv",
)

interactions["user_id"] = pd.to_numeric(interactions["user_id"],
errors="coerce").astype("Int64")
interactions["item_id"] = pd.to_numeric(interactions["item_id"],
errors="coerce").astype("Int64")
interactions["event"] = interactions["event"].astype(str).str.strip().str.lower()
interactions["event_weight"] = pd.to_numeric(interactions["event_weight"], errors="coerce")
interactions["event_ts"] = pd.to_datetime(interactions["event_ts"], errors="coerce")
interactions["event_date"] = pd.to_datetime(interactions["event_date"], errors="coerce")

interactions = interactions.dropna(subset=["user_id", "item_id", "event", "event_weight",
"event_ts"])
interactions["user_id"] = interactions["user_id"].astype("int64")
interactions["item_id"] = interactions["item_id"].astype("int64")

products.columns = [c.strip() for c in products.columns]

return interactions, products

def normalize_products(products: pd.DataFrame) -> pd.DataFrame:
    products = products.copy()

    if "item_id" not in products.columns:
        if "product_id" in products.columns:
            products["item_id"] = pd.to_numeric(products["product_id"], errors="coerce")
        elif "id" in products.columns:
            products["item_id"] = pd.to_numeric(products["id"], errors="coerce")

    if "item_id" not in products.columns:
        raise ValueError("products_prepared.csv must contain one of: item_id, product_id, or
id")

    products["item_id"] = pd.to_numeric(products["item_id"], errors="coerce")
    products = products.dropna(subset=["item_id"]).copy()
    products["item_id"] = products["item_id"].astype("int64")

    if "category" in products.columns:
        products["category"] = (
            products["category"]
            .astype(str)
            .str.strip()
            .str.lower()
        )
    else:
        products["category"] = "unknown"

    if "price_norm" not in products.columns:
        products["price_norm"] = min_max_scale(products["price"]) if "price" in
products.columns else 0.0

    if "rating_norm" not in products.columns:
        if "rating" in products.columns:
            products["rating_norm"] = min_max_scale(products["rating"])
        else:
            products["rating_norm"] = 0.0

    if "stock_norm" not in products.columns:
        if "stock" in products.columns:
            products["stock_norm"] = min_max_scale(products["stock"])
        else:
            products["stock_norm"] = 0.0

    keep_cols = ["item_id", "price_norm", "rating_norm", "stock_norm", "category"]
    return products[keep_cols].drop_duplicates(subset=["item_id"]).reset_index(drop=True)

def build_user_features(interactions: pd.DataFrame) -> pd.DataFrame:
    max_ts = interactions["event_ts"].max()

    user_features = (
        interactions.groupby("user_id")
        .agg(
            user_total_interactions=("user_id", "size"),
            user_view_count=("event", lambda s: (s == "view").sum()),
            user_addtocart_count=("event", lambda s: (s == "addtocart").sum()),

```

```

        user_transaction_count=("event", lambda s: (s == "transaction").sum()),
        user_avg_event_weight=("event_weight", "mean"),
        user_active_days=("event_date", lambda s: s.dt.date.nunique()),
        last_event_ts=("event_ts", "max"),
    )
    .reset_index()
)

user_features["user_last_seen_recency_days"] = (
    (max_ts - user_features["last_event_ts"]).dt.total_seconds() / 86400.0
)

user_features = user_features.drop(columns=["last_event_ts"])

return user_features

def build_item_features(interactions: pd.DataFrame, products: pd.DataFrame) -> pd.DataFrame:
    item_features = (
        interactions.groupby("item_id")
        .agg(
            item_total_interactions=("item_id", "size"),
            item_view_count=("event", lambda s: (s == "view").sum()),
            item_addtocart_count=("event", lambda s: (s == "addtocart").sum()),
            item_transaction_count=("event", lambda s: (s == "transaction").sum()),
            item_avg_event_weight=("event_weight", "mean"),
        )
        .reset_index()
    )

    item_features["item_popularity_rank"] = (
        item_features["item_total_interactions"]
        .rank(method="dense", ascending=False)
        .astype("int64")
    )

    products_norm = normalize_products(products)

    item_features = item_features.merge(products_norm, on="item_id", how="left")

    item_features["category"] = item_features["category"].fillna("unknown")
    for col in ["price_norm", "rating_norm", "stock_norm"]:
        item_features[col] = item_features[col].fillna(0.0)

    return item_features

def build_interaction_features(interactions: pd.DataFrame) -> pd.DataFrame:
    max_ts = interactions["event_ts"].max()

    interaction_features = (

```

Asset: *materialize_features.ipynb*

Path: src\04-features\materialize_features.ipynb

```

# Code cell 1
from __future__ import annotations

from pathlib import Path
import numpy as np
import pandas as pd

# -----
# Project paths and setup
# -----
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent
#PROJECT_ROOT = Path(__file__).resolve().parents[1]
PREP_ROOT = PROJECT_ROOT / "data" / "prepared"
FEATURE_ROOT = PROJECT_ROOT / "data" / "featured" / "v1"

INTERACTIONS_PATH = PREP_ROOT / "interactions_prepared.csv"
PRODUCTS_PATH = PREP_ROOT / "products_prepared.csv"

USER_FEATURES_PATH = FEATURE_ROOT / "user_features.parquet"
ITEM_FEATURES_PATH = FEATURE_ROOT / "item_features.parquet"
INTERACTION_FEATURES_PATH = FEATURE_ROOT / "interaction_features.parquet"

def min_max_scale(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors="coerce")
    if s.notna().sum() == 0:
        return pd.Series(np.nan, index=series.index)

    min_val = s.min()

```

```

max_val = s.max()

if pd.isna(min_val) or pd.isna(max_val):
    return pd.Series(np.nan, index=series.index)

if min_val == max_val:
    return pd.Series(0.0, index=series.index)

return (s - min_val) / (max_val - min_val)

def ensure_required_columns(df: pd.DataFrame, required: list[str], df_name: str) -> None:
    missing = [c for c in required if c not in df.columns]
    if missing:
        raise ValueError(f"{df_name} is missing required columns: {missing}")

def load_prepared_data() -> tuple[pd.DataFrame, pd.DataFrame]:
    if not INTERACTIONS_PATH.exists():
        raise FileNotFoundError(f"Missing file: {INTERACTIONS_PATH}")

    if not PRODUCTS_PATH.exists():
        raise FileNotFoundError(f"Missing file: {PRODUCTS_PATH}")

    interactions = pd.read_csv(INTERACTIONS_PATH)
    products = pd.read_csv(PRODUCTS_PATH)

    ensure_required_columns(
        interactions,
        ["user_id", "item_id", "event", "event_weight", "event_ts", "event_date"],
        "interactions_prepared.csv",
    )
    ensure_required_columns(
        products,
        ["category"],
        "products_prepared.csv",
    )

    interactions["user_id"] = pd.to_numeric(interactions["user_id"],
errors="coerce").astype("Int64")
    interactions["item_id"] = pd.to_numeric(interactions["item_id"],
errors="coerce").astype("Int64")
    interactions["event"] = interactions["event"].astype(str).str.strip().str.lower()
    interactions["event_weight"] = pd.to_numeric(interactions["event_weight"], errors="coerce")
    interactions["event_ts"] = pd.to_datetime(interactions["event_ts"], errors="coerce")
    interactions["event_date"] = pd.to_datetime(interactions["event_date"], errors="coerce")

    interactions = interactions.dropna(subset=["user_id", "item_id", "event", "event_weight",
"event_ts"])
    interactions["user_id"] = interactions["user_id"].astype("int64")
    interactions["item_id"] = interactions["item_id"].astype("int64")

    products.columns = [c.strip() for c in products.columns]

    return interactions, products

def normalize_products(products: pd.DataFrame) -> pd.DataFrame:
    products = products.copy()

    if "item_id" not in products.columns:
        if "product_id" in products.columns:
            products["item_id"] = pd.to_numeric(products["product_id"], errors="coerce")
        elif "id" in products.columns:
            products["item_id"] = pd.to_numeric(products["id"], errors="coerce")

    if "item_id" not in products.columns:
        raise ValueError("products_prepared.csv must contain one of: item_id, product_id, or
id")

    products["item_id"] = pd.to_numeric(products["item_id"], errors="coerce")
    products = products.dropna(subset=["item_id"]).copy()
    products["item_id"] = products["item_id"].astype("int64")

    if "category" in products.columns:
        products["category"] = (
            products["category"]
            .astype(str)
            .str.strip()
            .str.lower()
        )
    else:
        products["category"] = "unknown"

    if "price_norm" not in products.columns:
        products["price_norm"] = min_max_scale(products["price"]) if "price" in
products.columns else 0.0

    if "rating_norm" not in products.columns:

```

```

        if "rating" in products.columns:
            products["rating_norm"] = min_max_scale(products["rating"])
        else:
            products["rating_norm"] = 0.0

    if "stock_norm" not in products.columns:
        if "stock" in products.columns:
            products["stock_norm"] = min_max_scale(products["stock"])
        else:
            products["stock_norm"] = 0.0

    keep_cols = ["item_id", "price_norm", "rating_norm", "stock_norm", "category"]
    return products[keep_cols].drop_duplicates(subset=["item_id"]).reset_index(drop=True)

def build_user_features(interactions: pd.DataFrame) -> pd.DataFrame:
    max_ts = interactions["event_ts"].max()

    user_features = (
        interactions.groupby("user_id")
        .agg(
            user_total_interactions=("user_id", "size"),
            user_view_count=("event", lambda s: (s == "view").sum()),
            user_addtocart_count=("event", lambda s: (s == "addtocart").sum()),
            user_transaction_count=("event", lambda s: (s == "transaction").sum()),
            user_avg_event_weight=("event_weight", "mean"),
            user_active_days=("event_date", lambda s: s.dt.date.nunique()),
            last_event_ts=("event_ts", "max"),
        )
        .reset_index()
    )

    user_features["user_last_seen_recency_days"] = (
        (max_ts - user_features["last_event_ts"]).dt.total_seconds() / 86400.0
    )

    user_features = user_features.drop(columns=["last_event_ts"])

    return user_features

def build_item_features(interactions: pd.DataFrame, products: pd.DataFrame) -> pd.DataFrame:
    item_features = (
        interactions.groupby("item_id")
        .agg(
            item_total_interactions=("item_id", "size"),
            item_view_count=("event", lambda s: (s == "view").sum()),
            item_addtocart_count=("event", lambda s: (s == "addtocart").sum()),
            item_transaction_count=("event", lambda s: (s == "transaction").sum()),
            item_avg_event_weight=("event_weight", "mean"),
        )
        .reset_index()
    )

    item_features["item_popularity_rank"] = (
        item_features["item_total_interactions"]
        .rank(method="dense", ascending=False)
        .astype("int64")
    )

    products_norm = normalize_products(products)

    item_features = item_features.merge(products_norm, on="item_id", how="left")

    item_features["category"] = item_features["category"].fillna("unknown")
    for col in ["price_norm", "rating_norm", "stock_norm"]:
        item_features[col] = item_features[col].fillna(0.0)

    return item_features

def build_interaction_features(interactions: pd.DataFrame) -> pd.DataFrame:
    max_ts = interactions["event_ts"].max()

    interaction_features = (

```

Asset: 06 Feature Engineering and Transformation.ipynb

Path: src\07-submission\06 Feature Engineering and Transformation.ipynb

```

# Code cell 1
import sys
import subprocess
import json
import sqlite3
import shutil
import textwrap

```

```

from pathlib import Path
from datetime import datetime
from xml.sax.saxutils import escape

try:
    import pandas as pd
    import numpy as np

    from reportlab.lib import colors
    from reportlab.lib.pagesizes import A4
    from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
    from reportlab.lib.enums import TA_LEFT, TA_CENTER, TA_JUSTIFY
    from reportlab.lib.units import inch
    from reportlab.platypus import (
        SimpleDocTemplate,
        Paragraph,
        Spacer,
        Table,
        TableStyle,
        PageBreak,
        Preformatted,
    )
except ImportError:
    subprocess.check_call([
        sys.executable, "-m", "pip", "install",
        "pandas", "numpy", "reportlab", "pyarrow"
    ])
    import pandas as pd
    import numpy as np

    from reportlab.lib import colors
    from reportlab.lib.pagesizes import A4
    from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
    from reportlab.lib.enums import TA_LEFT, TA_CENTER, TA_JUSTIFY
    from reportlab.lib.units import inch
    from reportlab.platypus import (
        SimpleDocTemplate,
        Paragraph,
        Spacer,
        Table,
        TableStyle,
        PageBreak,
        Preformatted,
    )

# =====
# 1) PROJECT ROOT DISCOVERY
# =====
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent

RAW_ROOT = PROJECT_ROOT / "data" / "raw"
BRONZE_ROOT = PROJECT_ROOT / "data" / "bronze"
PREPARED_ROOT = PROJECT_ROOT / "data" / "prepared"
REPORTS_DIR = PROJECT_ROOT / "reports"
VALIDATION_DIR = REPORTS_DIR / "validation"
LOGS_DIR = PROJECT_ROOT / "logs"
SRC_DIR = PROJECT_ROOT / "src"

WAREHOUSE_DIR = PROJECT_ROOT / "warehouse"
WAREHOUSE_DIR.mkdir(parents=True, exist_ok=True)

TRANSFORMED_DIR = PROJECT_ROOT / "data" / "transformed"
TRANSFORMED_DIR.mkdir(parents=True, exist_ok=True)

OUTPUT_FILE_NAME = "06 Feature Engineering and Transformation- DM4ML-Group51.pdf"
OUTPUT_PATH = PROJECT_ROOT / OUTPUT_FILE_NAME

SQLITE_DB_PATH = WAREHOUSE_DIR / "recommendation_features.db"
SCHEMA_SQL_PATH = WAREHOUSE_DIR / "recommendation_features_schema.sql"

print(f"PROJECT_ROOT: {PROJECT_ROOT}")
print(f"RAW_ROOT: {RAW_ROOT}")
print(f"BRONZE_ROOT: {BRONZE_ROOT}")
print(f"WAREHOUSE DB: {SQLITE_DB_PATH}")
print(f"SCHEMA SQL: {SCHEMA_SQL_PATH}")
print(f"OUTPUT PDF: {OUTPUT_PATH}")

# =====
# 2) HELPERS
# =====
def safe_str(x):
    try:
        return str(x)
    except Exception:
        return ""

def rel_path(path):

```

```

    try:
        return safe_str(path.relative_to(PROJECT_ROOT))
    except Exception:
        return safe_str(path)

def collect_files(base_dir, patterns):
    results = []
    if not base_dir.exists():
        return results
    for pattern in patterns:
        results.extend(base_dir.rglob(pattern))
    return sorted(set(p for p in results if p.is_file()))

def latest_file(base_dir, patterns):
    files = collect_files(base_dir, patterns)
    if not files:
        return None
    return max(files, key=lambda p: p.stat().st_mtime)

def file_info(path):
    if not path or not path.exists():
        return None
    st = path.stat()
    return {
        "name": path.name,
        "relative_path": rel_path(path),
        "size_kb": round(st.st_size / 1024, 2),
        "modified": datetime.fromtimestamp(st.st_mtime).strftime("%Y-%m-%d %H:%M:%S"),
        "suffix": path.suffix.lower(),
    }

def read_text_preview(path, max_lines=60, max_chars=8000):
    if not path or not path.exists():
        return "File not found."
    try:
        text = path.read_text(encoding="utf-8", errors="ignore")
        lines = text.splitlines()[:max_lines]
        return "\n".join(lines)[:max_chars]
    except Exception as e:
        return f"Could not preview file: {e}"

def read_notebook_preview(path, max_code_cells=4, max_chars=7000):
    if not path or not path.exists():
        return "Notebook not found."
    try:
        with open(path, "r", encoding="utf-8") as f:
            nb = json.load(f)
            blocks = []
            code_idx = 0
            for cell in nb.get("cells", []):
                if cell.get("cell_type") != "code":
                    continue
                code_idx += 1
                src = cell.get("source", [])
                src = "".join(src) if isinstance(src, list) else str(src)
                src = src.strip()
                if src:
                    blocks.append(f"# Code cell {code_idx}\n{src}")
                if len("\n\n".join(blocks)) >= max_chars or code_idx >= max_code_cells:
                    break
            out = "\n\n".join(blocks).strip()
            return out[:max_chars] if out else "No code cells found."
    except Exception as e:
        return f"Could not parse notebook: {e}"

def read_code_preview(path, max_lines=120, max_chars=7000):
    if not path or not path.exists():
        return "File not found."
    if path.suffix.lower() == ".ipynb":
        return read_notebook_preview(path, max_code_cells=4, max_chars=max_chars)
    return read_text_preview(path, max_lines=max_lines, max_chars=max_chars)

def build_tree_text(base_path, max_depth=5, max_items=250):
    if not base_path.exists():
        return f"{base_path.name}/ (not found)"
    lines = [f"{base_path.name}/"]
    count = 0

    def walk(path, prefix="", depth=0):
        nonlocal count
        if depth >= max_depth or count >= max_items:
            return
        items = sorted(path.iterdir(), key=lambda p: (p.is_file(), p.name.lower()))
        for idx, item in enumerate(items):
            if count >= max_items:
                break
            connector = "■■■■" if idx == len(items) - 1 else "■■■■ "
            lines.append(prefix + connector + item.name + ("/" if item.is_dir() else ""))
            count += 1
        walk(item, prefix + connector, depth + 1)

    walk(base_path)
    return "\n".join(lines)

```



```

        if item.is_dir():
            extension = " " if idx == len(items) - 1 else "■ "
            walk(item, prefix + extension, depth + 1)

    walk(base_path)
    if count >= max_items:
        lines.append("... output truncated ...")
    return "\n".join(lines)

def wrap_block_text(text, width=95):
    wrapped = []
    for line in str(text).splitlines():
        if not line.strip():
            wrapped.append("")
            continue
        parts = textwrap.wrap(
            line,
            width=width,
            break_long_words=True,
            break_on_hyphens=True,
            replace_whitespace=False,
            drop_whitespace=False,
        )
        wrapped.extend(parts if parts else [""])
    return "\n".join(wrapped)

def make_hashable_value(v):
    if isinstance(v, np.ndarray):
        return tuple(make_hashable_value(x) for x in v.tolist())
    if isinstance

```

15. Validation / Pipeline Log Preview

```

{
  "cells": [
    {
      "cell_type": "code",
      "execution_count": 2,
      "id": "fef32579-f890-4b53-8d24-2fcae64a4914",
      "metadata": {},
      "outputs": [
        {
          "name": "stdout",
          "output_type": "stream",
          "text": [
            "PROJECT_ROOT: C:\\Users\\barath\\recomart-pipeline\\",
            "RAW_ROOT: C:\\Users\\barath\\recomart-pipeline\\data\\raw\\",
            "BRONZE_ROOT: C:\\Users\\barath\\recomart-pipeline\\data\\bronze\\",
            "\\n",
            "Discovered files:\\n",
            "- events_csv: C:\\Users\\barath\\recomart-  
pipeline\\data\\raw\\retailrocket\\load_date=2026-04-28\\load_hour=14\\events.csv\\n",
            "- category_tree_csv: C:\\Users\\barath\\recomart-  
pipeline\\data\\raw\\retailrocket\\load_date=2026-04-26\\load_hour=19\\category_tree.csv\\n",
            "- item_properties_part1_csv: C:\\Users\\barath\\recomart-pipeline\\data\\raw\\retailrock  
et\\load_date=2026-04-28\\load_hour=14\\item_properties_part1.csv\\n",
            "- item_properties_part2_csv: C:\\Users\\barath\\recomart-pipeline\\data\\raw\\retailrock  
et\\load_date=2026-04-28\\load_hour=14\\item_properties_part2.csv\\n",
            "- products_raw_json: C:\\Users\\barath\\recomart-  
pipeline\\data\\raw\\dummyjson\\load_date=2026-04-28\\load_hour=14\\products_raw.json\\n",
            "- categories_raw_json: C:\\Users\\barath\\recomart-  
pipeline\\data\\raw\\dummyjson\\load_date=2026-04-28\\load_hour=14\\categories_raw.json\\n",
            "- products_parquet: C:\\Users\\barath\\recomart-  
pipeline\\data\\bronze\\dummyjson\\products.parquet\\n",
            "- categories_parquet: C:\\Users\\barath\\recomart-  
pipeline\\data\\bronze\\dummyjson\\categories.parquet\\n",
            "\\n",
            "Initial status: FAIL\\n",
            "Final status: PASS\\n",
            "Initial issues file: C:\\Users\\barath\\recomart-  
pipeline\\reports\\validation\\validation_issues_initial_20260428T144031Z.csv\\n",
            "Initial summary file: C:\\Users\\barath\\recomart-  
pipeline\\reports\\validation\\validation_summary_initial_20260428T144031Z.csv\\n",
            "Revalidation issues file: C:\\Users\\barath\\recomart-  
pipeline\\reports\\validation\\validation_issues_revalidated_20260428T144031Z.csv\\n",
            "Revalidation summary file: C:\\Users\\barath\\recomart-  
pipeline\\reports\\validation\\validation_summary_revalidated_20260428T144031Z.csv\\n",
            "Fix log file: C:\\Users\\barath\\recomart-  
pipeline\\reports\\validation\\fix_log_20260428T144031Z.csv\\n",
            "JSON report file: C:\\Users\\barath\\recomart-  
pipeline\\reports\\validation\\data_quality_report_20260428T144031Z.json\\n",
            "PDF report file: C:\\Users\\barath\\recomart-  
pipeline\\reports\\validation\\data_quality_report_20260428T144031Z.pdf\\n"
          ]
        }
      ]
    }
  ],
}

```

```

"source": [
    "# DM4ML - Assignment - Validation + Auto-Fix + Revalidation + PDF Report\n",
    "\n",
    "import json\n",
    "from pathlib import Path\n",
    "from datetime import datetime, timezone\n",
    "\n",
    "import pandas as pd\n",
    "\n",
    "from reportlab.lib import colors\n",
    "from reportlab.lib.pagesizes import A4\n",
    "from reportlab.lib.styles import getSampleStyleSheet\n",
    "from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle,
PageBreak\n",
    "\n",
    "# =====\n",
    "# RecoMart Validation - robust file discovery + validation\n",
    "# with remediation, revalidation, and PDF reporting\n",
    "# =====\n",
    "\n",
    "PROJECT_ROOT = Path.cwd().resolve()\n",
    "while not (PROJECT_ROOT / \"src\").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:\n",
    "    PROJECT_ROOT = PROJECT_ROOT.parent\n",
    "\n",
    "RAW_ROOT = PROJECT_ROOT / \"data\" / \"raw\"\n",
    "BRONZE_ROOT = PROJECT_ROOT / \"data\" / \"bronze\"\n",
    "\n",
    "REMIEDIATED_RAW_ROOT = PROJECT_ROOT / \"data\" / \"remediated\" / \"raw\"\n",
    "REMIEDIATED_BRONZE_ROOT = PROJECT_ROOT / \"data\" / \"remediated\" / \"bronze\"\n",
    "\n",
    "REPORT_ROOT = PROJECT_ROOT / \"reports\" / \"validation\"\n",
    "LOG_ROOT = PROJECT_ROOT / \"logs\"\n",
    "\n",

```

16. Data Quality Report Preview

```

{
    "run_id": "20260429T111342Z",
    "run_ts": "2026-04-29T11:13:42.777701+00:00",
    "project_root": "C:\\Users\\barath\\recomart-pipeline",
    "initial_validation": {
        "run_id": "20260429T111342Z",
        "run_ts": "2026-04-29T11:13:42.777701+00:00",
        "phase": "initial",
        "project_root": "C:\\Users\\barath\\recomart-pipeline",
        "raw_root": "C:\\Users\\barath\\recomart-pipeline\\data\\raw",
        "bronze_root": "C:\\Users\\barath\\recomart-pipeline\\data\\bronze",
        "discovered_files": {
            "events_csv": "C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\retailrocket\\load_date=2026-04-29\\load_hour=11\\events.csv",
            "category_tree_csv": "C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\retailrocket\\load_date=2026-04-29\\load_hour=11\\category_tree.csv",
            "item_properties_part1_csv": "C:\\Users\\barath\\recomart-pipeline\\data\\raw\\retailrock
et\\load_date=2026-04-29\\load_hour=11\\item_properties_part1.csv",
            "item_properties_part2_csv": "C:\\Users\\barath\\recomart-pipeline\\data\\raw\\retailrock
et\\load_date=2026-04-29\\load_hour=11\\item_properties_part2.csv",
            "products_raw_json": "C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\dummyjson\\load_date=2026-04-29\\load_hour=11\\products_raw.json",
            "categories_raw_json": "C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\dummyjson\\load_date=2026-04-29\\load_hour=11\\categories_raw.json",
            "products_parquet": "C:\\Users\\barath\\recomart-
pipeline\\data\\bronze\\dummyjson\\products.parquet",
            "categories_parquet": "C:\\Users\\barath\\recomart-
pipeline\\data\\bronze\\dummyjson\\categories.parquet"
        },
        "overall_status": "FAIL",
        "issues_file": "C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\validation_issues_initial_20260429T111342Z.csv",
        "summary_file": "C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\validation_summary_initial_20260429T111342Z.csv",
        "datasets": [
            {
                "phase": "initial",
                "dataset": "retailrocket_events",
                "rows": 2756101,
                "columns": 5,
                "errors": 0,
                "warnings": 2734104,
                "status": "PASS"
            },
            {
                "phase": "initial",
                "dataset": "retailrocket_category_tree",
                "rows": 1669,
                "columns": 2,
                "errors": 0,
                "warnings": 25,

```

```

    "status": "PASS"
  },
  {
    "phase": "initial",
    "dataset": "retailrocket_item_properties",
    "rows": 20275902,
    "columns": 4,
    "errors": 0,
    "warnings": 0,
    "status": "PASS"
  },
  {
    "phase": "initial",
    "dataset": "dummyjson_products_raw",
    "rows": 194,
    "columns": 27,
    "errors": 0,
    "warnings": 92,
    "status": "PASS"
  }
}

```

17. Validation Event Log Preview

```

{"event_ts": "2026-04-29T11:13:42.785703+00:00", "stage": "validation", "status": "started",
"message": "Validation started", "run_id": "20260429T111342Z"}
{"event_ts": "2026-04-29T11:14:14.271184+00:00", "stage": "remediation", "status": "started",
"message": "Validation failed; starting remediation", "run_id": "20260429T111342Z"}
{"event_ts": "2026-04-29T11:17:44.036779+00:00", "stage": "validation", "status": "completed",
"message": "Validation workflow completed", "run_id": "20260429T111342Z", "initial_status":
"FAIL", "final_status": "PASS", "pdf_report_file": "C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\data_quality_report_20260429T111342Z.pdf"}

```

18. Conclusion

This PDF consolidates the project evidence for feature engineering and transformation, including recommendation features, SQL schema output, transformation asset discovery, and structured warehouse storage suitable for downstream training or feature serving.